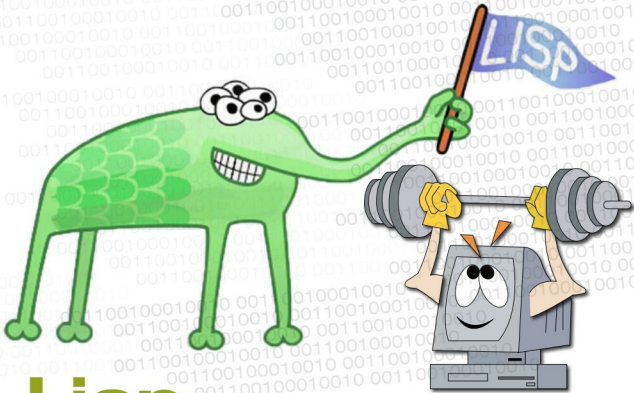




Lisp

Tears of Joy

Part—8

Lisp has been hailed as the world's most powerful programming language. But only the top percentile of programmers use it because of its cryptic syntax and academic reputation. This is rather unfortunate, since Lisp isn't that hard to grasp. If you want to be among the crème de la crème, this series is for you. This is the eighth article in the series that began in the June 2011 issue of LFY.

In rare moments of self-reflection, when I allow myself to doubt my skills as a Lisp evangelist, I sometimes wonder if I have left behind some of my fellow programmers who favour the object-oriented style of programming. Just because I have been focusing on Lisp as a functional programming paradigm, it doesn't mean we don't have a role for you in our plans of world domination. Read on to know where you fit in.

Functional vs object-oriented (OO) programming

With an OO approach, programmers write code that describes in exacting detail the steps that the computer must take to accomplish the goal. They focus on how to perform tasks, and how to track changes in state. They would use loops, conditions and method calls as their primary flow control, and instances of structures or classes as primary manipulation units. OO tries to control state behind object interfaces.

In contrast, functional programming (FP) involves composing the problem as a set of functions to be executed. FP programmers focus on what information is desired and what transformations are required, by carefully defining the input to each function and what each function returns. They would use function calls, including recursion, as their primary flow control, functions as first-class objects and data collections as primary manipulation units. FP tries to minimise state by using pure functions as much as possible.

According to Michael Feathers at <http://twitter.com/#!/mfeathers/status/29581296216>: "OO makes code understandable by encapsulating moving parts. FP makes code understandable by minimising moving parts."

Conard Barski points out that the critics of the OO programming style may complain that object-oriented techniques force data to be hidden away in a lot of disparate places by requiring them to live inside many different objects. Having data located in disparate places can make programs difficult to understand, especially if that data changes over